

ICML 2024 | Reaching a Goal Is Just Diffusion, Run in Reverse

McGill and Mila propose Merlin, a value-function-free method for goal-conditioned reinforcement learning

In offline goal-conditioned reinforcement learning, we want an agent to reach any target state from any starting point, learning purely from a batch of pre-collected data. The reward is usually sparse: 1 when the goal is reached, 0 otherwise. The setting is practical, and it is also stubbornly hard.

Most methods first learn a value function and then improve a policy against it. Offline, that first step is fragile. The policy proposes actions that never appear in the data, the value function scores those actions incorrectly, and the errors compound over time until the policy diverges. Sparse binary rewards only make the estimation problem harder.

This ICML 2024 paper, from McGill University and Mila, takes a different angle. If we treat goal states as the data distribution we want to model, is reaching a goal really so different from denoising in a diffusion model? Their method, Merlin, constructs trajectories that drift step by step away from the goal, then trains a policy to reverse that drift, with no value function anywhere in the loop.

Goal-conditioned Reinforcement Learning

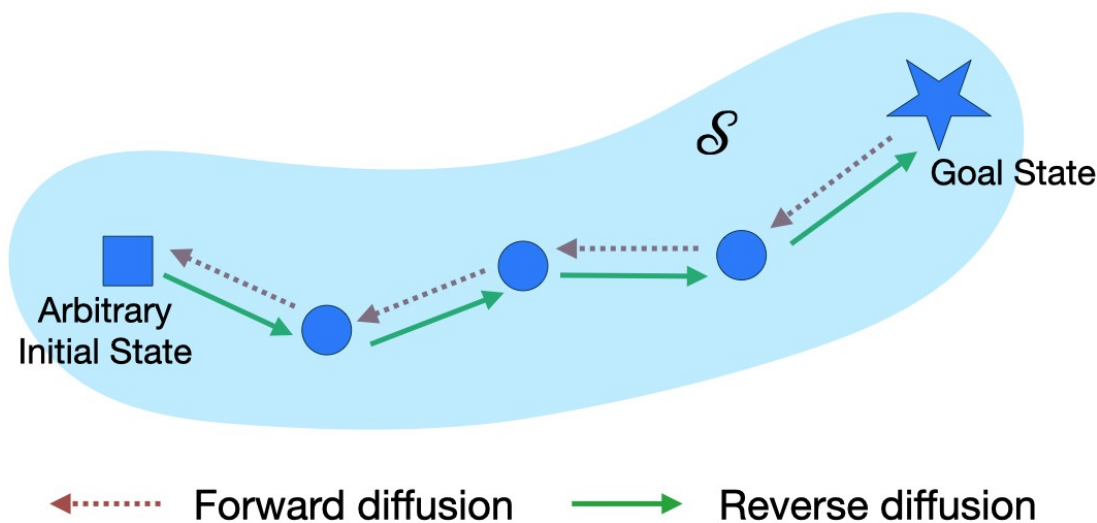


Figure 1. The core idea behind Merlin. Just as image diffusion turns Gaussian noise back into a real sample, Merlin builds trajectories that walk away from the goal (grey dotted arrows, forward diffusion), then learns a goal-conditioned policy to reverse them (green solid arrows, reverse diffusion), reaching the goal from an arbitrary initial state.

Paper: <https://arxiv.org/abs/2310.02505> | Code: <https://github.com/vineetjain96/merlin>

Why value functions struggle here

Online reinforcement learning can keep interacting with the environment and correct a bad value estimate. Offline, it cannot: once the dataset is fixed, the agent never receives new feedback. To stabilize offline learning, prior work typically constrains the policy or uses conservative value updates, but those fixes tend to cost performance and generalization. Underneath it all is a more basic question: do we actually need to estimate how good each state is at all?

Diffusion models are a working example of an answer that says no. A forward process gradually corrupts data into Gaussian noise, and a learned reverse process denoises it back into realistic samples, without ever estimating a value function. That analogy is exactly Merlin's starting point: noise is a walk away from the data manifold, and in goal-conditioned RL we can likewise construct trajectories that walk away from a goal, then learn to reverse them.

Merlin: goal-conditioned RL as reverse diffusion

Concretely: take a trajectory from the offline dataset and read it backwards, starting from its final goal state. Applying a forward diffusion transformation at each step produces a sequence of states that drift progressively away from the goal, exactly like adding noise in an image diffusion model. What Merlin learns is the goal-conditioned policy that reverses this drift, one step at a time.

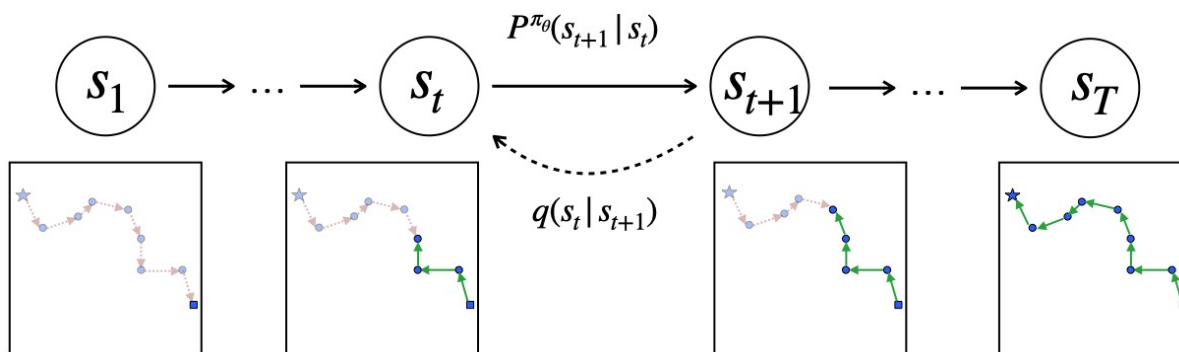


Figure 2. Forward and reverse diffusion for goal-conditioned RL, illustrated on a 2D navigation task. The star is the goal state; red dotted arrows are the forward transitions $q(s_t | s_{t+1})$, and green arrows the reverse transitions $P^{\pi_\theta}(s_{t+1} | s_t)$. The policy learns the green direction.

The key theoretical result is that maximizing the log-likelihood of goal states under this reverse process reduces to plain behavior cloning on hindsight-relabeled data. The value function drops out entirely, and the training objective collapses to a single supervised-learning expression. Just as important, because the diffusion happens directly in the state space, the policy needs only 1 denoising iteration per environment step, instead of the many iterations a typical diffusion policy runs. That is where Merlin's speed and stability come from.

Three ways to walk away from the goal

How you construct the forward trajectory determines what the policy has to reverse, and Merlin offers three constructions, one per variant. The base Merlin uses a fixed heuristic rule to

generate goal-departing trajectories; Merlin-P replaces the forward process with a learned parametric forward model; and Merlin-NP takes a non-parametric route, retrieving nearest neighbors in a learned latent space and stitching nearby states together.

That stitching is what makes Merlin-NP stronger. Because nearest neighbors are computed in the learned latent space, connected states are genuinely close in a meaningful sense, so the reverse policy can compose sub-segments from different trajectories into a new route to the goal, even one that never appears as a single trajectory in the data.

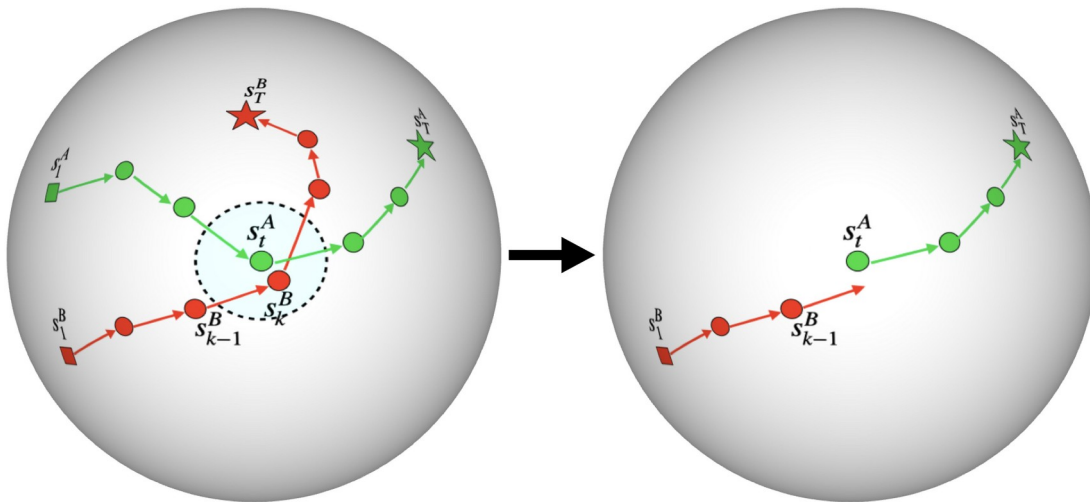


Figure 3. Trajectory stitching: nearest neighbors are computed in the learned latent space so that connected states sit close together (left), letting the reverse policy join two sub-trajectories into one coherent path to the goal (right). This lets Merlin-NP reach beyond the coverage of any single demonstration.

Does it work?

The authors evaluate on the offline goal-conditioned benchmark of Yang et al. (2021): 10 control tasks, from simple point navigation and Reacher to harder Fetch push, pick, and slide, plus the high-dimensional HandReach. Every task uses a sparse binary reward and a maximum trajectory length $T = 50$, and results are averaged across expert and random datasets over 10 random seeds. The baselines span goal-conditioned RL methods (GCSL, WGCSL, AM, GoFAR) and diffusion-based methods (Decision Diffuser, g-DQL, BES0).

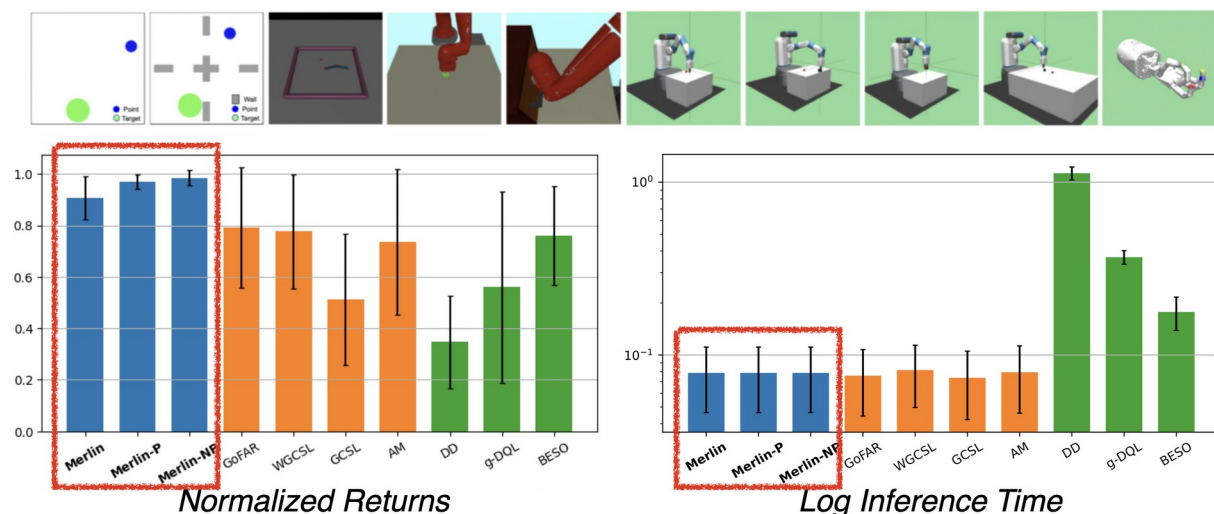


Figure 4. Benchmark tasks (top), normalized returns (left), and log inference time (right). Merlin, Merlin-P, and Merlin-NP (blue, boxed in red) match or exceed every baseline on returns while running about an order of magnitude (10×) faster than the diffusion methods (DD, BESO).

The picture is consistent. Base Merlin already beats the baselines on most tasks, and Merlin-P and Merlin-NP push returns higher still. Measured by average rank across all 10 methods and 10 tasks, Merlin-NP comes out on top, with an average rank around 1.7 on state observations and about 1.25 on pixel observations. And because it performs a single denoising step per environment step, its training and inference run roughly 10× faster than diffusion methods like Decision Diffuser and BESO. The higher-dimensional the observation, such as pixel input, the larger that speed gap grows.

Table 1. The three Merlin variants and how each constructs its forward (goal-departing) trajectory.

Variant	Forward-trajectory construction
Merlin	Fixed heuristic rule
Merlin-P	Learned parametric forward model
Merlin-NP	Non-parametric nearest-neighbor stitching in latent space

How far ahead to aim

Merlin has two main tunable hyperparameters: the hindsight-relabeling ratio and the evaluation time horizon h , which sets how far ahead the policy aims at each step. The authors sweep h systematically and plot the outcome as heatmaps.

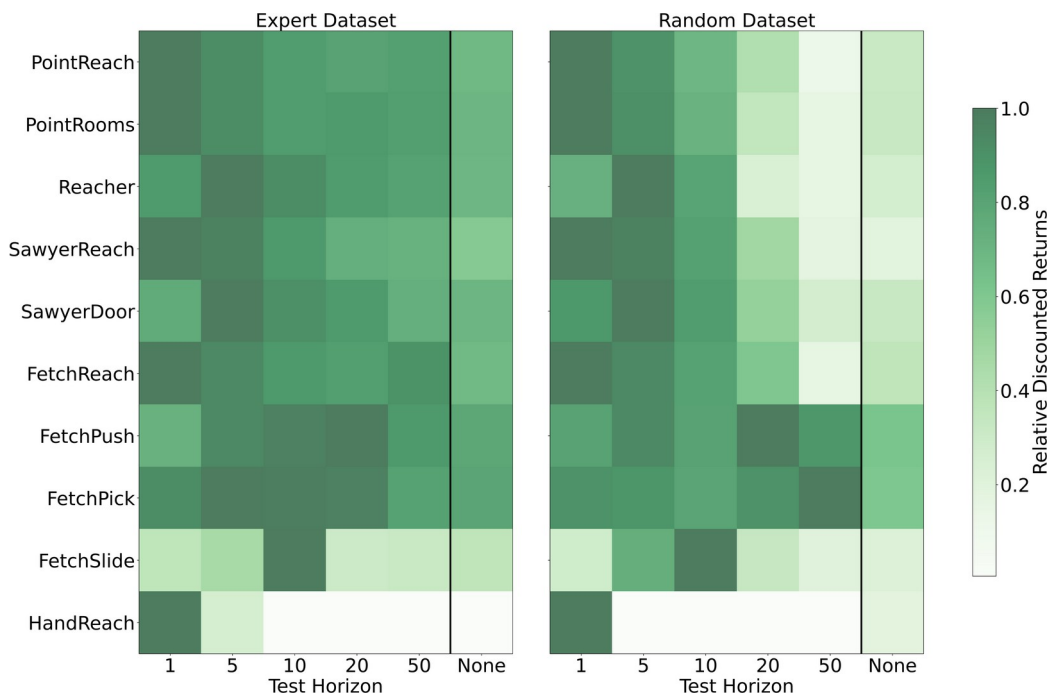


Figure 5. Relative discounted returns per task across evaluation horizons h (left: expert dataset; right: random dataset). Conditioning on some horizon consistently beats not conditioning (the None column), and the best h shifts with task difficulty: easier tasks favor short horizons, and HandReach in particular works well only at $h=1$.

Two things stand out. First, conditioning on a time horizon almost always beats not conditioning at all (the rightmost None column is visibly lighter). Second, the best horizon depends on the task: an easy task like PointReach does well at a short horizon ($h=1$ or $h=5$), while harder tasks need longer ones, with the sweep spanning roughly $\{1, 5, 10, 20, 50\}$. HandReach is the most sensitive of all, working only at $h=1$. So Merlin is broadly robust to the horizon choice, though a few high-dimensional tasks still need task-specific tuning.

Takeaway and limits

The point worth remembering is that Merlin rewrites offline goal-conditioned RL as the reverse of a diffusion process, and that rewrite makes the problem remarkably simple. Construct trajectories that walk away from goals, learn to reverse them, and goal-reaching collapses into ordinary behavior cloning, with no value function to estimate and no attendant instability to fight. It matches or beats state-of-the-art methods across 10 tasks while running about an order of magnitude faster than other diffusion-based approaches.

The framework does come with conditions. It targets the offline, sparse-reward, goal-conditioned setting; the choice of forward-trajectory construction (fixed, parametric, or non-parametric stitching) has to be matched to the task, and a few high-dimensional tasks such as HandReach remain sensitive to hyperparameters like the horizon. But performing diffusion directly in the state space, with a single denoising step per environment step, is what makes diffusion for RL look, for the first time, both simple and scalable, and that may matter more than any single benchmark number.